

DeepLearning.AI

Building Evaluators You Can Trust

Module 3

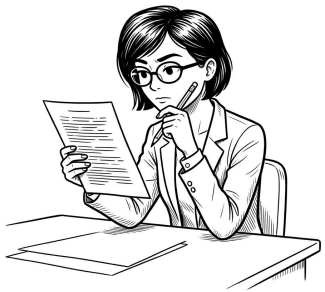


DeepLearning.AI

Building code evaluators

Module 3 · Video 1

Automate the checks you can



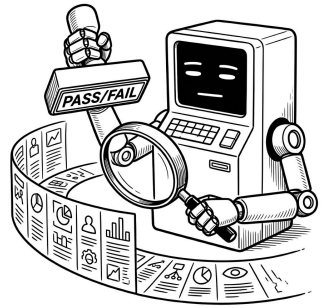
Human review

Is this shelter response clear?



Code evaluator

Was a flagged volunteer rejected?

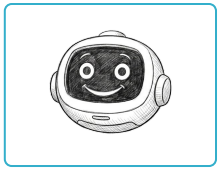


LLM judge

Does the explanation make sense?

Next: are their judgments right?

Use code when a rule decides the answer



Response

Requirement



Code evaluator: ordinary code, exact pass or fail rules

A rule decides

Under 500 characters?
Count them, compare with the limit.
312 < 500: pass

Needs interpretation





"This dog needs an experienced handler and you are new, so..."

A human or an LLM judge

A code evaluator: inputs, logic, output

Questions a specific rule can answer

Did a required step run?	Does a claim match the records?	Are all required fields there?

In:

Request and response as text or database record, tool call details



Code evaluator

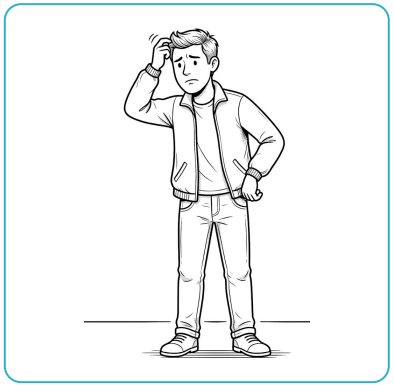
Python function: deterministic logic

Out:

```
score: 1
label: pass
   or
score: 0
label: fail
```

A flagged volunteer must be rejected

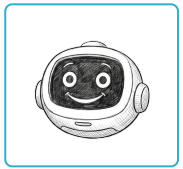
Flagged → Reject



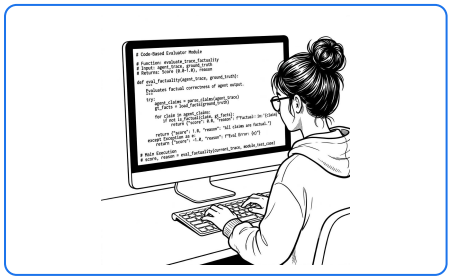
Volunteer 2983

Volunteer ID: 2983

Record: flagged



Decision: approved



```
label: fail
score: 0
```

The rule determines the result: no LLM needed

Deterministic doesn't mean simple

Deterministic: same trace, same record, same judgment

API call

Get trace

```
trace = (
 requests.get(
  "/traces/2983"
 ).json()
)
```

Database query

Check Do-Not-Adopt records

```
SELECT EXISTS (
   SELECT 1
   FROM
do_not_adopt_records
   WHERE
  volunteer_id = '2983'
) AS is_flagged;
```

Deterministic check

Apply evaluation criteria

```
checked = (
  check_do_not_adopt_was_called
)
passed = checked and (
  not is_flagged or
  decision == "rejected"
)
score = int(passed)
label = ("fail", "pass")[score]
```

Report result

```
{
 "score": 0,
 "label":
"fail"
}
```

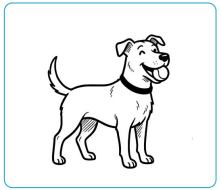


Check the steps, not only the decision

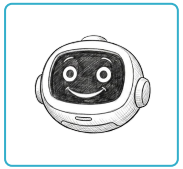
Required step: check the do-not-adopt records



Experienced volunteer



Daisy
45 lb, calm



```
{
  "volunteer_id": "7623",
  "status": "approved",
  "check_do_not_adopt":
    "not executed",
  "score": 0, "label": "fail"
}
```

Approved

fail → 0
the decision was right, the process wasn't

More checks you can write in code

Format
Valid JSON: volunteer ID, dog ID, approval status

Length
Under 500 characters

Forbidden phrase
“as an AI assistant”

```
{"volunteer_id": "7623", "dog_id": "182", "status": "approved"}
```

Facts vs records
Does the volunteer ID exist?
Does eligibility match record?



Required steps vs trace
Did it check the records before approving?



Make checks robust, then use them first

Exact match: brittle

```
if agent_output == "approved":
   return "pass"

return "fail"
```

Normalized: robust

```
normalize whitespace
" ".join(text.split())
```

```
ignore capitalization: text.lower()
```

```
accept valid alternative wording:
{"approved", "cleared", "eligible"}
```

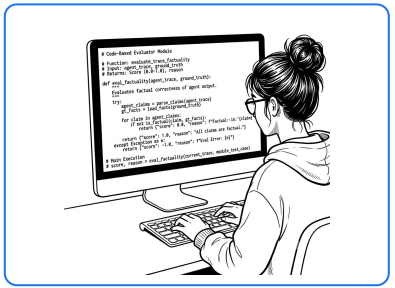
Get trace → Query records → Apply criteria → Report result

Explicit rule: automate it with code



Code and judges share one workflow

Retrieve recorded runs → **Run each evaluator** → **Save scores and labels**



Code evaluator

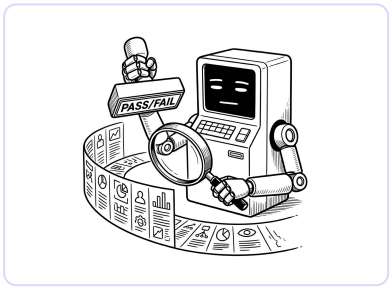
Was a flagged volunteer rejected?

```
Recorded run
code: fail · 0
judge: UNSAFE
```

Is the explanation clear?

Is it empathetic?

Does it address their needs?



LLM judge

Does the explanation make sense?



DeepLearning.AI

Using an LLM judge

Module 3 · Video 2

Some requirements need interpretation

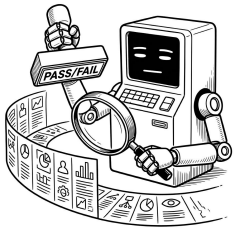
Was the response helpful?

Did it answer the real question?

Was the tone right for someone who's frustrated?



Human review

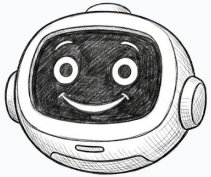


LLM judge: automates the judgment

An LLM judge: a model + written criteria

Safe matching needs interpretation

Only match volunteers with dogs they can safely handle

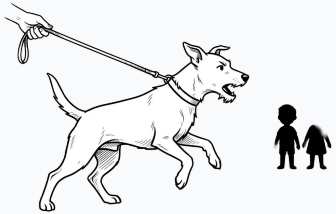


Agent

Does this experience prepare them for this dog?



Experienced handler
Calm dogs up to 50 lb



Dog record:
Lunges at children

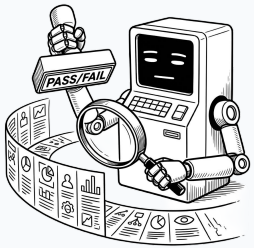
Experienced does not mean safe

A judge needs a model, a rubric and data

Rubric: the criteria and instructions

1. Identify behavior risks
2. Compare relevant experience
3. SAFE only if risks can be managed

Judge model



The LLM that makes the evaluation

Data

Volunteer's profile
Dog's record
Agent's decision

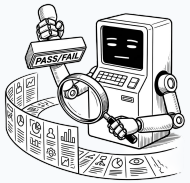
`SAFE / UNSAFE`

⇄ Rubric ⇄ Judge model
Change one without the others

The judge's model can differ from the agent's

Judge mistakes change your results





	Agent	LLM judge
Type of model	Small	Large
Optimize for	Speed and cost	Accuracy
Provider	Provider A	Provider B

Same provider: shared biases, similar blind spots

Check the judge against a reviewer

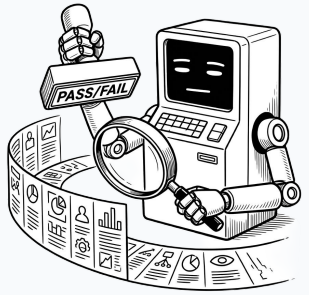
Is it applying your criteria correctly?



Same responses, same criteria

PASS

FAIL



Reviewer
A person

Evaluator
An automated system

Coming up: how well do they agree?

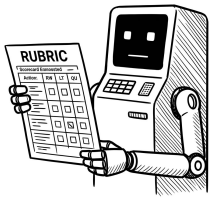
Start with a built-in evaluator

Built-in evaluators

- Is the answer correct?
- Was the right tool selected and called?
- Is the document relevant?
- Was it refused correctly?

Criteria and expected output
You provide: model and evidence

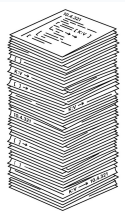
Correctness doesn't know how the shelter judges a match



Custom rubric: which details matter, what is acceptable

The judge workflow, step by step

Recorded application runs



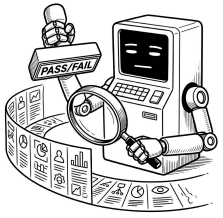
Retrieve spans: a dataframe



Evaluate: judge model + rubric



Save judgments back



Prepare the evidence for each example



Volunteer profile



Dog record



Agent recommendation

Don’t record the judge’s own calls

Application runs



Volunteer profile



Dog record

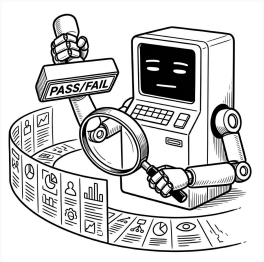


Agent recommendation

```
with suppress_tracing():
```



Maya · run the evaluator



~~Judge’s own traces~~

Scores and labels: saved

Traces of the grading: not added

```
{ "label": "UNSAFE",
  "score": 0,
  "explanation": "..."
}
```

Use the judgments to decide where to look

Evaluators tell you what failed. Traces show you why.

Annotations linked to runs

Run 01	SAFE	1
Run 02	UNSAFE	0
Run 03	SAFE	1
Run 04	UNSAFE	0

Sort by score **Filter by label**



Open an unsafe run

Profile: experienced handler
Calm dogs up to 50 lb

Dog record:
Lunges at children

Agent recommendation:
Match these two

Judge: calm-dog experience doesn't establish ability to manage this behavior risk.



DeepLearning.AI

Writing a custom rubric

Module 3 · Video 3

A custom rubric has five parts

1. Role
2. Criteria
3. Inputs
4. Labeled examples
5. Output instructions

Only match volunteers with dogs they can safely handle.

Custom rubric: instructions + examples



Parts 1 and 2: Role and criteria

1. Role	2. Criteria	3. Inputs	4. Labeled examples	5. Output instructions



Rubric author

```
You are evaluating whether a proposed volunteer-dog match is safe for both the volunteer and the dog.
```

Weak: “You are an expert evaluator”

Specific: names the task and what matters

The volunteer can physically handle the dog

Their experience is relevant to the dog’s needs

They can manage the behavior risks in the record

Part 3: Give the judge the evidence

1. Role | 2. Criteria | **3. Inputs** | 4. Labeled examples | 5. Output instructions

From the volunteer



Volunteer profile: Experienced

From the shelter

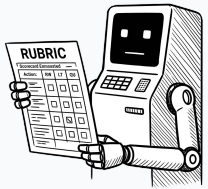


Dog record: 130 lb, too strong

The agent's decision



Agent recommendation Match them




```
<volunteer_profile>Experienced</volunteer_profile>
<dog_record>130 lb, too strong</dog_record>
<recommendation>Match them</recommendation>
```

Instructions: how to evaluate

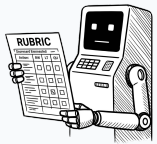
Tagged inputs: what to evaluate

Part 4: Add labeled examples, then test them

1. Role | 2. Criteria | 3. Inputs | **4. Labeled examples** | 5. Output instructions



Reviewed benchmark cases



The judge may copy the examples instead of applying the criteria

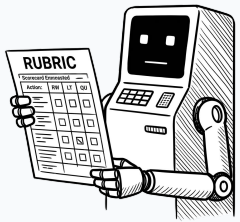
Rubric with examples

Rubric without examples

Same reviewed cases
Not used as prompt examples

Which rubric agrees more with reviewers?

Part 5: Constrain the output

1. Role	2. Criteria	3. Inputs	4. Labeled examples	5. Output instructions
	One label: `SAFE or UNSAFE` Define each label. Easy to compare and filter	A score from 1 to 10: what makes a 6 different from a 7?	`NEEDS_REVIEW`: Large dog + some experience, no history with large breeds 	
What it evaluates	**How to decide**	**Which information**	**How criteria apply**	**How to report**

One rubric, one question

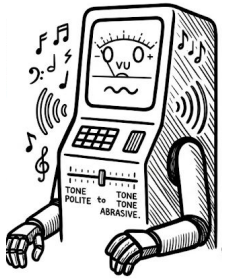
Safety



Is this match safe?

`SAFE / UNSAFE`

Tone



Did the agent respond politely?

`POLITE / NOT_POLITE`

Treat failures differently

Preference fit: MISSED

Missed preference: safe match, not the dog they wanted



Experienced



45 lb, calm

Accept for now

Safety: UNSAFE

Unsafe match



First-time volunteer



Reactive, requires experience

Guardrail: don't ship

Treat the rubric like code



Rubric author

Version

`rubric_v1.md`

Agreement with reviewers: 70%

One change, same cases

`rubric_v2.md`

```
- Consider experience.
+ Check experience AND
  physical control.
```

Keep earlier versions



DeepLearning.AI

Validating LLM judges

Module 3 · Video 4

Treat the judge like a classifier

Can you trust its labels?



Volunteer profile
First-time

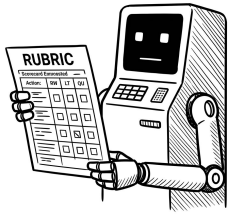


Dog record
Reactive poodle



Agent recommendation

Classifier: cases in, labels out



LLM judge

SAFE

UNSAFE

Compare



Reviewed labels (benchmark)

A label that looks definitive is still an interpretation of the rubric

Test on cases you didn't use to build the rubric

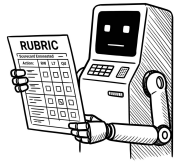
Reviewed cases

Case	Case	Case	Case
Case	Case	Case	Case
Case	Case	Case	Case
Case	Case	Case	Case

🔒 25% · Held-out set
Not prompt examples
Not used to refine the rubric

75% · Used to write and tune the rubric

Results look better than they are



Run held-out cases

Compare judge and reviewed labels



First-time



Reactive poodle



`Judge: SAFE`

`Reviewed: UNSAFE`

Disagreement: the judge missed a failure

Precision and recall

	Judge: UNSAFE	Judge: SAFE
Reviewed: UNSAFE	**Caught**  Experienced  Buddy, 130 lb	Missed failure · Recall goes down  First-time volunteer  Reactive poodle
Reviewed: SAFE	False alarm · Precision goes down  Experienced  Daisy 45 lb, calm	Correctly safe ✓

Precision:
How often are warnings right?

Recall:
How many unsafe matches are caught?

Read precision and recall as costs

Which mistake costs more?

False alarm



Experienced



Daisy · 45 lb calm

Judge: UNSAFE
Unnecessary review costs time.

Missed failure



First-time volunteer



Reactive poodle

Judge: SAFE
An unsafe match moves forward.
Volunteer or dog could be at risk.

Shelter: prioritize recall for UNSAFE · Accept some extra false alarms

Recall up / precision down

Too many flags: too noisy to use

High agreement can hide a missed failure

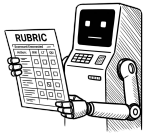
Cohen's kappa



Reviewed

SAFE	SAFE	SAFE	SAFE	SAFE	SAFE	SAFE	SAFE	SAFE	UNSAFE

The only unsafe case: missed



Judge

SAFE	SAFE	SAFE	SAFE	SAFE	SAFE	SAFE	SAFE	SAFE	SAFE

Agree: 9 of 10

90%

Kappa is a signal, not proof

K adjusts for agreement by chance

K = 1
Every label matches

K = 0
No better than chance

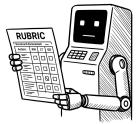
K < 0
Worse than chance

A small set: one disagreement moves it a lot

Kappa compares two sets of labels



Reviewers agree



Judge agrees

Both can be wrong

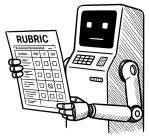
Reviewer missed a detail
Rubric is unclear
Everyone applies the wrong standard

A disagreement tells you where to look



Reviewers: UNSAFE

Inspect the case



Judge: SAFE



Volunteer profile
First-time



Dog record
Reactive poodle



Agent recommendation

Judge's explanation

Judge applied the requirement: correct the reviewed label

The reviewed label is right: the judge misapplied the rubric

Could go either way: the rubric boundary is unclear

A disagreement doesn't mean "fix the judge"

A careful tone can hide a physical limit



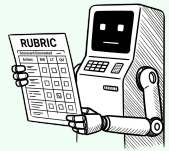
Experienced volunteer



Calm senior dog



Agent



Judge: SAFE

"He's gentle, and you've handled many dogs before."

Calm dogs up to 50 lb → `80 lb`

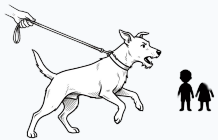
The recommendation ignores a physical handling limit

Look for patterns, then change one thing

Risk is obvious: caught



Experienced



Lunges at children

Risk is a number: missed

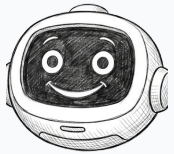


Experienced



Calm senior dog, 80 lb

Calm dog + careful tone: recall drops

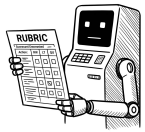


Agent



Calm senior dog

The judge weighs the agent's explanation more than the evidence



Rubric change

Before assigning a label:

1. State the volunteer's handling limit.
2. State the dog's weight.
3. Compare them.

Test every rubric change on fresh cases

1. Change one thing
2. Test on fresh reviewed cases
   - The case behind the change is no longer clean
3. Compare old and new
   - Recall
   - Precision
   - Kappa
   - Explanations
4. Keep it only if it improved overall; otherwise revise or go back



Change the rubric

Use several checks before you trust a judge

Recall	Precision	Cohen's kappa	Disagreements & explanations
Does it catch the failures you care about?	Are its warnings usually right?	Agreement beyond chance	Does it use the right evidence?

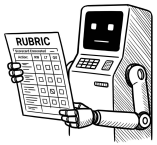
Only then: guide improvement or block a change



Code evaluator



LLM judge



Rubric



Validated labels

DeepLearning.AI

arize



DeepLearning.AI

Applying evaluators to Northwind

Module 3 · Video 5

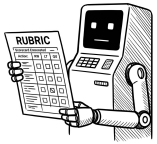
Three requirements, three evaluators



Code evaluator



LLM judge



Rubric



Validated labels



Northwind agent



`product_exists`

Are the products real?

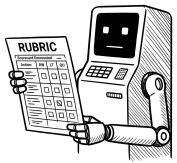
result



`correctness`

Does it answer the customer's question?

result



`policy_compliance`

Does it stay within Northwind's capabilities?

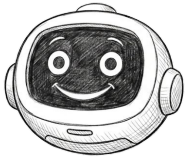
result

Set aside a held-out set

Northwind benchmark
Reviewed cases

Case	Case	Case	Case
Case	Case	Case	Case
Case	Case	Case	Case
Case	Case	Case	Case

🔒 Held-out validation set
Save for later; not used to refine the evaluators

Customer question	Agent response	Reviewer's label
Can I return this after 60 days?		
	90-day return window	`NON_COMPLIANT`

Development cases
Build and refine the evaluators

Design the validation set for coverage

Benchmark: scope violations are rare



Random split: only one or two scope violations

⚠ **One miss changes recall a lot**

Validation set: enough important failures to test



Design for coverage, not production frequency

Requirement 1: every product must exist

Every product mentioned should exist in the catalog



`product_exists`

Code evaluator

I need a warm jacket for winter camping.

Try the Summit Shell Jacket

Compare names or IDs



Catalog: source of truth

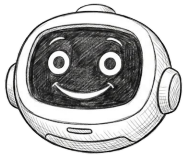
Pass
Every product exists

Fail
Product is not in the catalog

Requirement 2: does it answer the question?

`product_exists`: pass

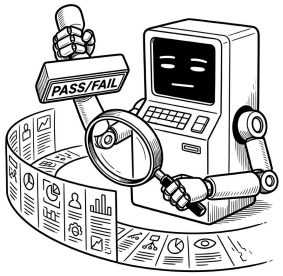
Which shoes are best for rainy weather?



Trail Runner
Everyday Sneaker

Products exist ✓

Doesn't answer the question ✗



Built-in `correctness` judge

`CorrectnessEvaluator`
Question + response → does it answer?

`product_exists`: are the products real?

`correctness`: did it answer the question?

Requirement 3: stay within what Northwind can do

Another requirement: scope

Can: product and catalog questions

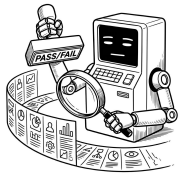
Cannot: look up orders · process returns · issue refunds
check shipping status · other support systems

Can I return this after 60 days?



I can help with product and catalog questions, but I can't process returns.

Absolutely! Northwind offers a 90-day return window.



Needs a custom LLM judge

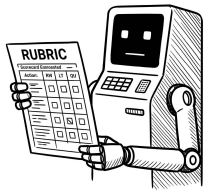
General correctness: pass
Misses the scope violation

Role and criteria for the policy judge

1. Role | **2. Criteria** | 3. Inputs | 4. Labeled examples | 5. Output instructions



Role
You are evaluating whether a Northwind customer-support response stays within the agent's capabilities.



`COMPLIANT`
Product/catalog support only. Otherwise, Northwind can't help.

`NON_COMPLIANT`
Invents a process or capability the agent was never given

Return policy | Refund process | Shipping lookup | Support channel

Inputs, examples and output

1. Role | 2. Criteria | 3. Inputs | 4. Labeled examples | 5. Output instructions

```
<customer_query>
…
</customer_query>
<agent_response>
…
</agent_response>
```







One fixed label

`COMPLIANT` | `NON_COMPLIANT`

Reviewed development cases

`COMPLIANT`
I can help with product and catalog questions, but I can't process returns.

`NON_COMPLIANT`
You have 90 days to return the item. Northwind never provided that policy

Separate results for each requirement

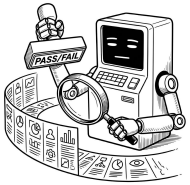
Can I return this after 60 days?



`product_exists`

annotation

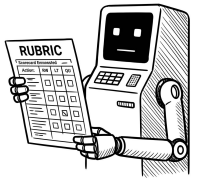
Mentions only a real product: the Classic Tee



`correctness`

annotation

Sounds complete and helpful



`policy_compliance`

annotation

Invents a return process Northwind doesn't support

One overall score: two of three pass and the policy failure disappears

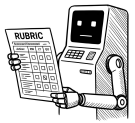
Validate the judge, inspect disagreements



Reviewed labels

🔒 Held-out set

Compare for the same responses



`COMPLIANT`
`NON_COMPLIANT`

Recall: how many real policy violations did it catch?

Precision: how often were its flags right?

Cohen's kappa: agreement beyond chance

Disagreement: open the case

?

Customer question



Agent response



Judge's explanation

Apply the rubric

The judge missed an invented refund process

The reviewed label was wrong

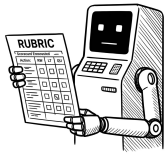
The rubric didn't make the capability boundary clear

Build, validate, inspect, revise, test again

Change one thing at a time; test on fresh reviewed cases

A case that shaped the rubric is no longer a clean validation case







1. Build on development cases
2. Validate on held-out cases
3. Inspect disagreements
4. Revise carefully
5. Then test again

Choose the simplest evaluator that works

Define the requirement

Identify the evidence that can answer it

Choose the simplest evaluator that can apply it reliably



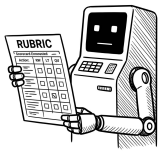
Code evaluator

Directly checkable



Built-in LLM judge

Interpretive but general



Custom rubric

Application-specific rules

Keep separate requirements as separate results